1. DevOps

1.1 ¿Qué significa DevOps, para qué sirve y cómo funciona?

DevOps es la unión de las palabras *Development* (desarrollo) y *Operations* (operaciones). Es una cultura, un conjunto de prácticas y una forma de organizar el trabajo que busca que los equipos que **desarrollan** el software y los que lo **operan** (infraestructura, servidores, monitoreo) trabajen juntos durante todo el ciclo de vida de una aplicación, en lugar de funcionar como áreas separadas que se pasan el trabajo "por encima de la pared".

¿Para qué sirve?

- Entregar software más rápido y con mayor frecuencia.
- Reducir errores en los despliegues y el tiempo de recuperación cuando algo falla.
- Automatizar tareas repetitivas (compilar, probar, desplegar, configurar servidores).
- Mejorar la comunicación y la responsabilidad compartida entre desarrolladores y operaciones.
- Obtener feedback rápido de los usuarios y del monitoreo para mejorar el producto de forma continua.

¿Cómo funciona? DevOps se representa habitualmente como un ciclo infinito (∞) de fases que se repiten constantemente:

1. **Plan (Planificar):** se definen las tareas y requerimientos.
2. **Code (Codificar):** los desarrolladores escriben el código y lo guardan en un repositorio con control de versiones.
3. **Build (Construir):** el código se compila y empaqueta de forma automática.
4. **Test (Probar):** se ejecutan pruebas automáticas (unitarias, integración, seguridad).
5. **Release (Liberar):** se prepara la versión aprobada para ser entregada.
6. **Deploy (Desplegar):** la versión se instala en el entorno destino (por ejemplo, producción).
7. **Operate (Operar):** se administra la infraestructura y la aplicación en funcionamiento.
8. **Monitor (Monitorear):** se recolectan métricas y logs para detectar problemas; esa información vuelve a la fase de planificación.

Los pilares que lo sostienen son: **cultura de colaboración, automatización, medición (métricas) y compartir conocimiento**.

1.2 Relación con las metodologías Ágiles y diferencias

Relación. Las metodologías ágiles (Scrum, Kanban, XP) nacieron para mejorar la forma en que los equipos **desarrollan** software: iteraciones cortas, entregas incrementales, adaptación al cambio y contacto cercano con el cliente. DevOps **continúa esa filosofía más allá del desarrollo**: extiende la agilidad hacia la entrega y la operación. Un equipo ágil puede terminar una funcionalidad cada dos semanas, pero si desplegarla en producción demora meses por procesos manuales, se pierde el beneficio; DevOps resuelve ese cuello de botella. Por eso se dice que ambos son **complementarios**.

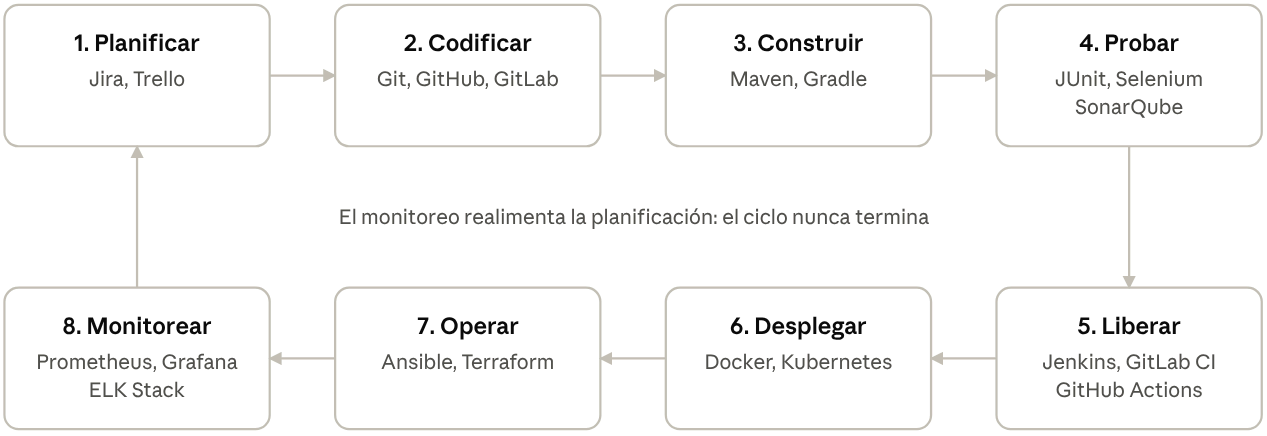
Diferencias principales:

Aspecto	Metodologías Ágiles	DevOps
Foco principal	Cómo se planifica y desarrolla el software	Cómo se entrega, despliega y opera el software
Alcance	Equipo de desarrollo y cliente / producto	Desarrollo + operaciones + seguridad + QA
Objetivo	Adaptarse al cambio y entregar valor en iteraciones cortas	Automatizar y acelerar el ciclo completo hasta producción y su monitoreo
Ejemplos de prácticas	Sprints, historias de usuario, daily, retrospectivas	CI/CD, infraestructura como código, monitoreo, contenedores
Ejemplos de marcos/herramientas	Scrum, Kanban, XP, Jira, Trello	Git, Jenkins, Docker, Kubernetes, Terraform, Prometheus
Entregable típico	Incremento de producto funcional al final del sprint	Pipeline automatizado que lleva el cambio a producción de forma segura

1.3 Gráfico de DevOps y herramientas sugeridas

Gráfico 1. Ciclo DevOps y herramientas por fase (elaboración propia, generado por Claude).

DevOps es un ciclo continuo: cada fase tiene sus herramientas



ciclo DevOps · 8 fases y sus herramientas

Las fases se leen de izquierda a derecha en la fila superior y de derecha a izquierda en la inferior.

Herramientas por fase y su explicación:

Fase	Herramienta	¿Qué hace?
Plan	**Jira / Trello**	Gestión de tareas, backlog y sprints; permite seguir el avance del equipo.
Code	**Git + GitHub / GitLab / Bitbucket**	Git es el sistema de control de versiones; GitHub, GitLab y Bitbucket alojan los repositorios y facilitan la colaboración (pull requests, revisiones de código).
Build	**Maven / Gradle**	Compilan el proyecto Java/Spring Boot, gestionan dependencias y generan el archivo `.jar`.
Test	**JUnit, Mockito, Selenium, SonarQube**	JUnit y Mockito para pruebas unitarias; Selenium para pruebas de interfaz; SonarQube analiza la calidad y seguridad del código.

Fase	Herramienta	¿Qué hace?
Integración / Entrega	**Jenkins / GitHub Actions / GitLab CI**	Automatizan el pipeline: ante cada cambio compilan, prueban y despliegan.
Empaquetado	**Docker**	Empaqueta la aplicación y sus dependencias en un contenedor portable.
Orquestación	**Kubernetes**	Administra, escala y recupera automáticamente los contenedores en producción.
Infraestructura como código	**Terraform / Ansible**	Terraform crea la infraestructura (servidores, redes) mediante archivos de configuración; Ansible configura y mantiene los servidores de forma automatizada.
Monitoreo	**Prometheus + Grafana, ELK Stack**	Prometheus recolecta métricas y Grafana las visualiza en tableros; ELK (Elasticsearch, Logstash, Kibana) centraliza y analiza los logs.
Comunicación	**Slack / Microsoft Teams**	Canales de comunicación y alertas automáticas del pipeline y del monitoreo.

2. Ambiente de Producción y Despliegue en Producción

2.1 ¿Qué significa que una aplicación se despliega en un entorno de Producción?

Desplegar (*deploy*) una aplicación en **producción** significa instalarla y ponerla en funcionamiento en el entorno **real**, accesible para los **usuarios finales**, con datos reales y con la expectativa de que funcione de manera estable, segura y disponible las 24 horas. Es el último paso del proceso: lo que antes era código en la computadora de un desarrollador pasa a ser un servicio vivo en internet (por ejemplo, en un servidor o en la nube, con un dominio y certificado HTTPS).

2.2 Diferencias entre entorno de desarrollo, pruebas y producción

Característica	Desarrollo (DEV)	Pruebas / Testing (QA / Staging)	Producción (PROD)
Propósito	Escribir y probar código mientras se construye	Verificar que el software funciona antes de liberarlo	Brindar el servicio real a los usuarios
Quién lo usa	Desarrolladores	Testers / QA, a veces el cliente (aceptación)	Usuarios finales
Infraestructura	Computadora local o servidor pequeño	Réplica lo más parecida posible a producción	Servidores robustos, a veces con balanceo de carga y alta disponibilidad
Datos	Datos ficticios o de ejemplo	Datos de prueba (o copia anonimizada)	Datos reales de los usuarios
Estabilidad	Cambia constantemente, puede fallar	Relativamente estable	Debe ser muy estable
Seguridad	Básica	Intermedia	Máxima (HTTPS, firewall, accesos restringidos, backups)
Errores	Se muestran con detalle para depurar	Se registran y se reportan	Se ocultan al usuario y se registran en logs
Monitoreo	Mínimo	Moderado	Completo con alertas

En una aplicación Spring Boot esto suele manejarse con **perfiles** (`application-dev.properties` , `application-test.properties` , `application-prod.properties`), que cambian la base de datos, las credenciales y el nivel de logs según el entorno.

2.3 Dominio de internet

¿Qué es? Un dominio es el nombre legible por personas que se usa para ubicar un sitio o servicio en internet (por ejemplo `google.com`). Reemplaza a la dirección IP numérica (como `142.250.78.14`), que es difícil de recordar.

¿Para qué sirve? Para identificar y acceder fácilmente a un sitio, construir una marca, y crear correos corporativos (`nombre@miempresa.com`). Un dominio se compone de partes: `www.ejemplo.com.ar` → subdominio (`www`), nombre (`ejemplo`) y extensión o TLD (`.com.ar`).

¿Cómo funciona? Cuando el usuario escribe un dominio en el navegador, la computadora consulta un servidor DNS que traduce el nombre a una dirección IP; con esa IP el navegador se conecta al servidor web que contiene el sitio y recibe la página.

5 ejemplos de dominios conocidos:

1. `google.com` (buscador)
2. `wikipedia.org` (enciclopedia)
3. `mercadolibre.com.ar` (comercio electrónico, Argentina)
4. `youtube.com` (videos)
5. `argentina.gob.ar` (sitio del Estado argentino)

2.4 Servidor DNS (Sistema de Nombres de Dominio)

El **DNS** (*Domain Name System*) es el sistema que funciona como la "agenda telefónica de internet": traduce nombres de dominio a direcciones IP. Un **servidor DNS** es la computadora que almacena y responde esas consultas.

Proceso simplificado de una consulta:

1. El usuario escribe `ejemplo.com.ar` en el navegador.
2. El navegador consulta primero su caché y la del sistema operativo.
3. Si no lo encuentra, pregunta al **resolver DNS** (normalmente el del proveedor de internet, o uno público como 8.8.8.8 de Google o 1.1.1.1 de Cloudflare).
4. El resolver consulta a los servidores **raíz**, luego a los del **TLD** (`.ar`) y finalmente al **servidor autoritativo** del dominio, que conoce la IP.
5. La IP vuelve al navegador, que se conecta al servidor web.

En los servidores DNS se configuran **registros**, por ejemplo: **A** (nombre → IPv4), **AAAA** (nombre → IPv6), **CNAME** (alias de un nombre a otro), **MX** (servidores de correo) y **TXT** (verificaciones y seguridad).

2.5 ¿Cómo obtener un dominio?

En Argentina (`.ar` , `.com.ar` , `.gob.ar` , etc.): los dominios `.ar` los administra **NIC Argentina** (nic.ar), dependiente de la Secretaría Legal y Técnica de la Presidencia de la Nación. El trámite se hace online: se crea una cuenta con CUIT/CUIL y clave fiscal (AFIP/ARCA), se busca la disponibilidad del nombre, se registra y se configuran los servidores DNS (los de NIC o los de un proveedor de hosting). Los dominios `.ar` tienen un período de registro con renovación; conviene verificar en nic.ar los requisitos y el costo vigente.

En el resto del mundo (`.com` , `.net` , `.org` , `.io` , etc.): se compran a través de un **registrador acreditado** por ICANN. Se busca el nombre, se paga una suscripción anual y se configuran los DNS. Ejemplos de registradores: GoDaddy, Namecheap, Cloudflare Registrar, Google/Squarespace Domains y Hostinger. Los precios típicos de un `.com` rondan los 10–20 USD por año, aunque varían según el proveedor y las promociones.

¿Cómo obtener un dominio gratuito?

- **Subdominios gratuitos** que ofrecen las plataformas de hosting: por ejemplo `miapp.onrender.com` , `miapp.vercel.app` , `miapp.netlify.app` , `miusuario.github.io` (GitHub Pages) o `miapp.herokuapp.com` según el plan disponible. No son dominios propios, sino subdominios del proveedor.
- **Dominio gratis el primer año** incluido en ciertos planes de hosting anuales (promoción de varios proveedores).
- **Servicios de DNS dinámico** gratuitos como No-IP o DuckDNS, que entregan un subdominio (`miapp.duckdns.org`).
- **Programa GitHub Student Developer Pack**, que para estudiantes suele incluir un dominio gratuito por un año en algunos proveedores (sujeto a las condiciones vigentes del programa).

Los dominios gratuitos son útiles para prácticas y proyectos académicos, pero para un sistema real conviene un dominio propio.

2.6 Servidor web y hosting

¿Qué es un servidor web? Es un software (y, por extensión, la computadora donde corre) que recibe peticiones HTTP/HTTPS de los clientes (navegadores, aplicaciones móviles) y devuelve respuestas: páginas, archivos o datos (por ejemplo JSON de una API). Ejemplos de software: **Apache HTTP Server, Nginx, Apache Tomcat** (contenedor de servlets Java)

y **IIS**. En Spring Boot, el servidor **Tomcat viene embebido** dentro del `.jar`, por lo que la aplicación se ejecuta con `java -jar` sin instalar un Tomcat aparte; a menudo se coloca **Nginx como proxy inverso** delante para manejar HTTPS y balanceo.

¿Qué significa hacer Hosting de una aplicación? Hosting (alojamiento) es contratar un servicio que provee un servidor conectado a internet 24/7 donde se aloja la aplicación, su base de datos y sus archivos, para que sea accesible públicamente. Tipos habituales:

- **Hosting compartido:** muchos sitios en un mismo servidor. Económico, pero limitado; en general **no sirve para Spring Boot**, porque no permite ejecutar procesos Java propios con control de puertos y memoria.
- **VPS (Servidor Privado Virtual) / Cloud Server:** máquina virtual con recursos dedicados y acceso root (SSH). Es la opción más común para Spring Boot.
- **Servidor dedicado:** una máquina física completa para un solo cliente.
- **PaaS (Plataforma como servicio):** el proveedor administra la infraestructura y uno solo sube la aplicación (Render, Railway, Heroku, Google App Engine, AWS Elastic Beanstalk).
- **Contenedores / Kubernetes administrado:** servicios que ejecutan imágenes Docker (AWS ECS/EKS, Google Cloud Run/GKE, Azure Container Apps/AKS).

2.7 Don Web: características y costos para aplicaciones Spring Boot

Don Web es una empresa de hosting y servicios cloud de origen argentino (Rosario, Santa Fe) con presencia regional en Latinoamérica. Ofrece hosting compartido, dominios, **Cloud Servers / VPS**, servidores dedicados y servicios de infraestructura en la nube.

Características relevantes para Spring Boot:

- Una aplicación Spring Boot **requiere un Cloud Server / VPS** (acceso root por SSH), no un hosting compartido, para poder instalar el JDK, ejecutar el `.jar` o contenedores Docker, abrir puertos y configurar un proxy inverso (Nginx) y HTTPS (Let's Encrypt).
- Los Cloud Servers se ofrecen con distintas configuraciones de vCPU, RAM y almacenamiento SSD, con sistemas operativos Linux (Ubuntu, Debian, CentOS, etc.) y Windows, y la posibilidad de escalar recursos.
- Como regla práctica, para una aplicación Spring Boot pequeña conviene al menos **1–2 vCPU y 2 GB de RAM**, ya que la JVM consume bastante memoria. Un servidor de este estilo puede llegar a costar $17.400 por mes.
- Dominios `.ar` y `.com` pueden gestionarse en el mismo proveedor, junto con la zona DNS.

Costos: los precios de Don Web se publican en pesos argentinos y dólares, **cambian con frecuencia** (inflación, tipo de cambio, promociones por pago anual) y varían según la configuración elegida. Durante la búsqueda realizada para esta investigación no se pudo obtener una tabla de precios oficial y actual, y algunos agregadores indican que el sitio original fue migrado, por lo que **no se consignan valores concretos**. Se recomienda verificar los planes de *Cloud Server / VPS* directamente en el sitio oficial y completar acá el valor vigente al momento de entregar el trabajo.

Dato a completar	Valor
Plan Cloud Server elegido (vCPU / RAM / SSD)	*completar*
Precio mensual	*completar*
Precio de dominio `.com.ar` / `.com`	*completar*
Fecha de consulta	*completar*

2.8 Servidores web / proveedores de hosting internacionales

- **Amazon Web Services (AWS):** EC2 (servidores virtuales), Lightsail (VPS simple), Elastic Beanstalk (PaaS para Java), ECS/EKS (contenedores).
- **Google Cloud Platform (GCP):** Compute Engine, App Engine, Cloud Run, GKE.
- **Microsoft Azure:** Virtual Machines, App Service (soporta Java/Spring Boot), AKS.
- **DigitalOcean:** Droplets (VPS) simples y económicos, App Platform y Kubernetes administrado.
- **Hetzner, Linode (Akamai), Vultr:** VPS con buena relación precio/rendimiento.
- **Hostinger, IONOS, A2 Hosting, Liquid Web:** hosting con planes VPS compatibles con Java.
- **Render, Railway, Fly.io, Heroku:** plataformas PaaS orientadas a desarrolladores, con despliegue desde Git o Docker.

2.9 Docker

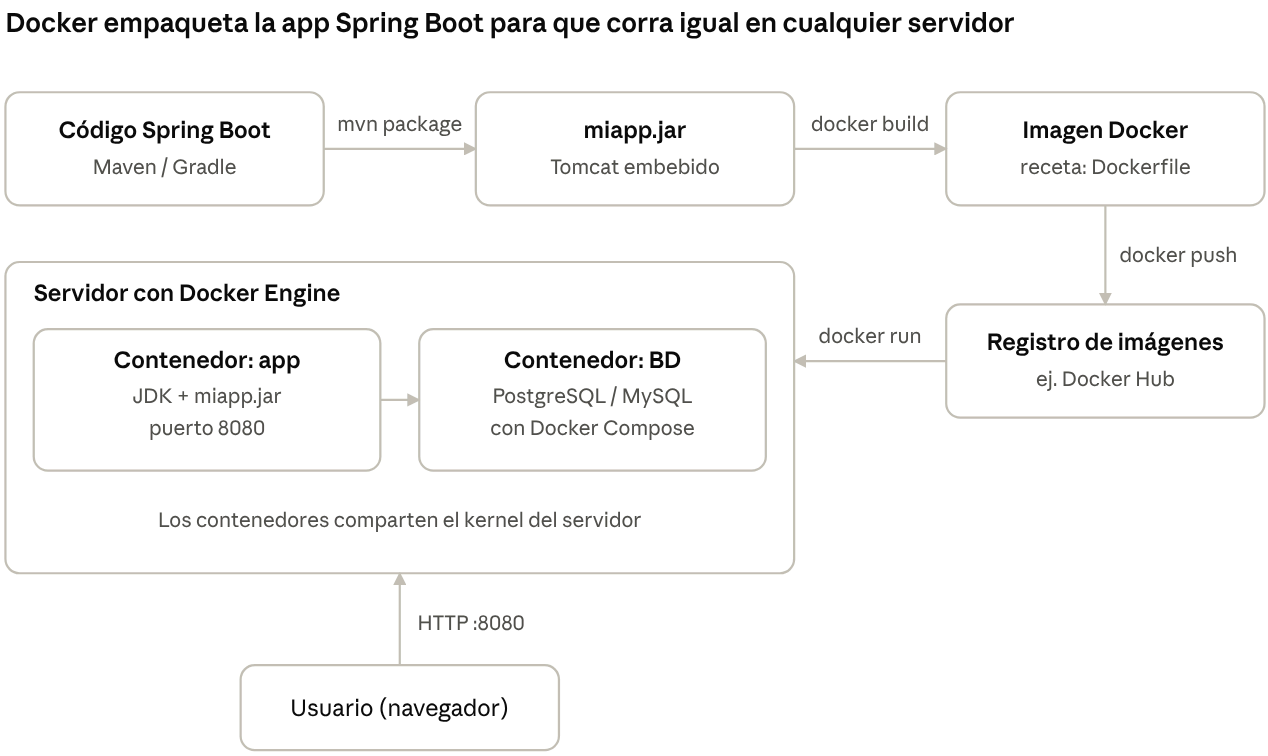
¿Qué es y para qué sirve? Docker es una plataforma que permite empaquetar una aplicación junto con todo lo que necesita para ejecutarse (sistema base, JDK, librerías, configuración) en una unidad llamada **contenedor**. Resuelve el clásico problema de *"en mi*

máquina funciona": el mismo contenedor se ejecuta igual en desarrollo, pruebas y producción. Los contenedores son más livianos que las máquinas virtuales porque comparten el kernel del sistema operativo anfitrión.

Conceptos clave: **Dockerfile** (receta para construir la imagen), **imagen** (plantilla inmutable de la aplicación), **contenedor** (instancia en ejecución de una imagen), **registro** (repositorio de imágenes como Docker Hub) y **Docker Compose** (define y levanta varios contenedores juntos, por ejemplo la app y la base de datos).

Funcionamiento para una aplicación Spring Boot:

Gráfico 2. Docker con una aplicación Spring Boot (elaboración propia, generado por Claude).



Docker + Spring Boot · del código al contenedor

El `.jar` se convierte en una imagen, la imagen se guarda en un registro y cualquier servidor con Docker la ejecuta como contenedor.

Ejemplo de `Dockerfile` para Spring Boot:

```
FROM eclipse-temurin:21-jre
WORKDIR /app
COPY target/miapp.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Comandos básicos: `docker build -t miapp .` (construye la imagen) y `docker run -p 8080:8080 miapp` (ejecuta el contenedor).

2.10 Kubernetes

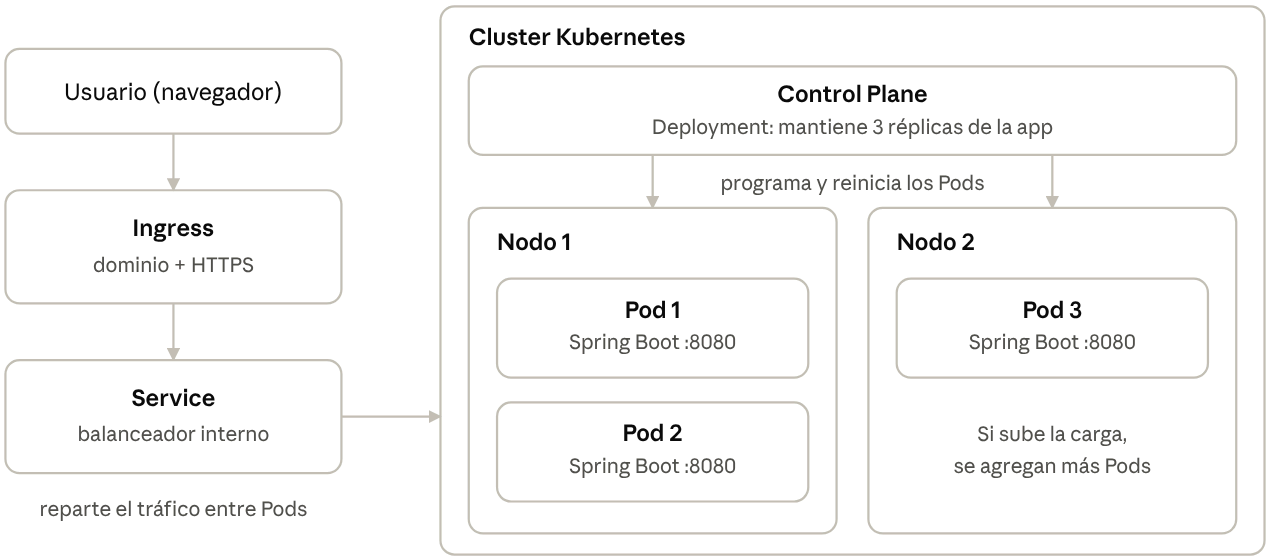
¿Qué es y para qué sirve? Kubernetes (abreviado **K8s**) es una plataforma de código abierto, creada originalmente por Google, que **orquesta contenedores**: decide en qué máquinas se ejecutan, los reinicia si fallan, los escala según la demanda, balancea la carga entre ellos y permite actualizar versiones sin cortar el servicio (*rolling updates*). Es necesario cuando hay muchos contenedores o se requiere alta disponibilidad.

Conceptos clave: **Cluster** (conjunto de máquinas), **Nodo** (máquina que ejecuta contenedores), **Pod** (unidad mínima, uno o más contenedores), **Deployment** (define cuántas réplicas del Pod y cómo actualizarlas), **Service** (punto de acceso estable y balanceador interno), **Ingress** (expone la aplicación a internet con dominio y HTTPS), **ConfigMap / Secret** (configuración y credenciales) y **Control Plane** (cerebro del cluster).

Funcionamiento para una aplicación Spring Boot:

Gráfico 3. Kubernetes con una aplicación Spring Boot (elaboración propia, generado por Claude).

Kubernetes reparte el tráfico entre Pods de Spring Boot y los mantiene vivos



Kubernetes + Spring Boot · cluster, nodos y Pods

El tráfico entra por el Ingress, el Service lo reparte entre los Pods y el Control Plane se encarga de mantener las réplicas activas.

2.11 Diferencias entre Docker y Kubernetes

Aspecto	Docker	Kubernetes
Función	Crear, empaquetar y ejecutar contenedores	Orquestar y administrar muchos contenedores en varias máquinas
Nivel	Un contenedor / un host	Un cluster de múltiples nodos
Escalado	Manual (o limitado con Compose/Swarm)	Automático según carga (autoscaling)
Recuperación ante fallos	Limitada	Automática: reinicia y reprograma Pods caídos
Balanceo de carga	Básico	Integrado (Service / Ingress)
Complejidad	Baja, fácil de aprender	Alta, requiere más conocimientos
Uso ideal	Desarrollo, pruebas y apps pequeñas	Producción a gran escala, microservicios

No son competidores: **Docker construye los contenedores y Kubernetes los gestiona**. Normalmente se usan juntos.

3. Integración Continua y Despliegue Continuo (CI/CD)

3.1 ¿Qué significan CI y CD?

Integración Continua (CI – *Continuous Integration*): práctica por la cual los desarrolladores integran sus cambios al repositorio compartido con mucha frecuencia (varias veces al día). Cada integración dispara automáticamente la **compilación** y las **pruebas automáticas**, de modo que los errores se detectan enseguida y no al final del proyecto.

Entrega Continua (CD – *Continuous Delivery*): extiende la CI: cada cambio que pasa las pruebas queda **listo para desplegarse** en producción, pero la liberación final requiere una **aprobación manual**.

Despliegue Continuo (CD – *Continuous Deployment*): va un paso más allá: si el cambio pasa todas las pruebas automáticas, se **despliega automáticamente a producción** sin intervención humana.

	Integración Continua	Entrega Continua	Despliegue Continuo
Automatiza	Compilar y probar	Compilar, probar y dejar listo el paquete	Todo, hasta producción
Paso a producción	No incluido	Manual (con un botón de aprobación)	Automático

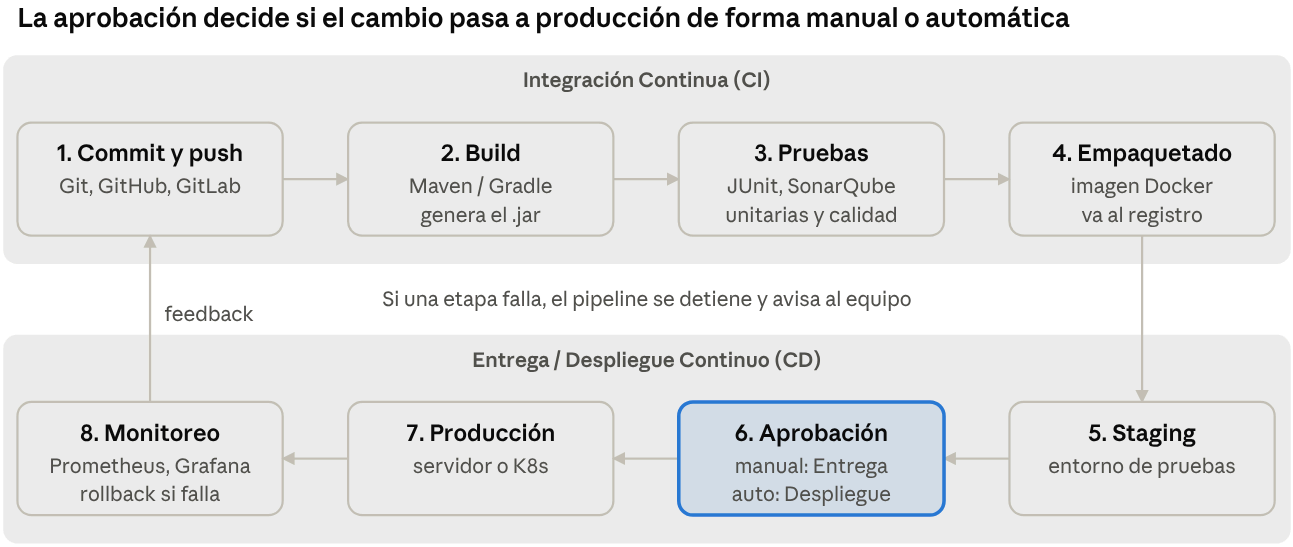
3.2 Ventajas

- **Detección temprana de errores:** se corrigen cuando son pequeños y baratos de arreglar.
- **Menos conflictos de integración**, porque los cambios son chicos y frecuentes.
- **Entregas más rápidas y frecuentes** de nuevas funcionalidades.
- **Despliegues más seguros y repetibles:** se elimina el error humano de los pasos manuales.
- **Mayor calidad del software** gracias a pruebas automáticas constantes.

- **Feedback rápido** para el equipo y los usuarios.
- **Menor tiempo de recuperación:** si algo falla, se puede volver a una versión anterior (*rollback*) con facilidad.
- **Ahorro de tiempo y costos** al automatizar tareas repetitivas.

3.3 Proceso de CI/CD en el desarrollo de una aplicación

Gráfico 4. Proceso de CI/CD en el desarrollo de una aplicación (elaboración propia, generado por Claude).



pipeline CI/CD · 8 etapas, 1 aprobación

La fila superior es la Integración Continua; la inferior, la Entrega o el Despliegue Continuo, y el monitoreo devuelve feedback al desarrollador.

Descripción de las etapas del pipeline (ejemplo con Spring Boot):

1. **Commit / Push:** el desarrollador sube su cambio al repositorio Git (GitHub/GitLab).
2. **Disparador (trigger):** el servidor de CI (Jenkins, GitHub Actions, GitLab CI) detecta el cambio.
3. **Build:** se compila con Maven/Gradle y se genera el `.jar`.
4. **Pruebas automáticas:** unitarias (JUnit), de integración y análisis de calidad (SonarQube).
5. **Empaquetado:** se construye la imagen Docker y se publica en un registro.
6. **Despliegue en Staging:** se instala en el entorno de pruebas para validación.
7. **Aprobación:** manual (Entrega Continua) o automática (Despliegue Continuo).

8. **Despliegue en Producción:** se actualiza el servidor o el cluster Kubernetes.
9. **Monitoreo y feedback:** se vigilan métricas y logs; si hay fallos, se hace rollback y se informa al equipo.

4. Fuentes y referencias

- Gráficos 1 a 4: **elaboración generada por Claude (Anthropic)**.
- Documentación oficial de Docker: https://docs.docker.com
- Documentación oficial de Kubernetes: https://kubernetes.io/docs
- Documentación de Spring Boot: https://spring.io/projects/spring-boot
- NIC Argentina (registro de dominios .ar): https://nic.ar
- ICANN (registradores de dominios): https://www.icann.org
- Atlassian – DevOps y CI/CD: https://www.atlassian.com/devops y https://www.atlassian.com/continuous-delivery
- Información sobre Don Web: sitio oficial del proveedor.